Operációs rendszerek

1. gyakorlat

Alapvető tudnivalók

Bemutatkozás

Gyakorlatvezető:	Klettner Péter
Honlap:	klettner.web.elte.hu
	(probléma esetén ld. GYIK)
E-Mail:	klettner@inf.elte.hu
Szoba:	déli épület 4. emelet 4.711

Tematika

1. C nyelvi ismeretek felelevenítése, kiegészítése
2. Párhuzamos folyamatok
3. Bináris fájlkezelés
4. Folyamatvezérlés, szinkronizálás
5. Folyamatok közötti adatcsere
6. Folyamatok közös memóriahasználata

Tudnivalók, követelmények

Elvárt előismeretek:

- *Számítógépes rendszerek*
  (Putty és WinSCP, UNIX parancsok, Szignálok)
- *Programozás (algoritmusok)*
- *Imperatív programozás (C programozási nyelv)*

Tudnivalók, követelmények

Gyakorlat időpontok (tervezett):

3. csop.:	Csütörtök	10:05- 10:50
4. csop.:	Csütörtök	9:15- 10:00
15. csop.:	Csütörtök	12:15- 13:00
16. csop.:	Csütörtök	13:05- 13:50
esti 3. csop.:	Csütörtök	18:35- 19:20

Hiányzások: HKR szerint

Konzultáció: Csütörtök 11:00-
Csütörtök 15:30-
előzetes egyeztetés alapján
(ld. klettner.web.elte.hu:
JELENTKEZÉS KONZULTÁCIÓRA)

Tudnivalók, követelmények

Tárgy honlapja:

opsys.inf.elte.hu

Követelmény:

ld. tárgy honlapja és előadás canvas

Osztályzat kiszámítási módja:

ha ZH1<2 vagy ZH2<2 vagy beadandó nincs elfogadva: **1**, különben: **átlag(ZH1,ZH2)-(beadandó késések)±(1)**,

ahol a ±1 értékét a beadandók igényessége és a gyakorlatokon végzett munka minősége határozza meg

Beadandók

- Feladatleírás:
  - előadás canvas
- Beadás:
  - canvas feltöltés
  - <neptunkód>.zip állomány
  - ne legyen a tömörített fájlban további mappa
  - legyen mellékelve adatfájl a teszteléshez, amennyiben a feladat indokolja

Beadandók

- Elfogadás feltételei:
  - Fordítás és tesztelés az opsys.inf.elte.hu szerveren.
  - A programkód egyetlen C nyelvű kódot tartalmazó fájl.
  - Fordítás: *gcc forrásfájl*.
  - Error és warning mentes fordítás (nem –w kapcsoló).
  - Teljes funkcionalitás a feladat szövege szerint.
  - Hibátlan működés.
  - Követhető működés.
  - Az Op. rend. tárgyból tanult megoldások alkalmazása.
  - Saját, önálló munka (nem internet, nem csapatmunka).
  - Határidők betartása.
  - Bemutatás.

Szerver

PuTTY:

- Host: opsys.inf.elte.hu
- login as: {inf-es azonosító}
- password: {inf-es jelszó}

VIM

vim {fájlnév} vagy
vi {fájlnév}

- i INSERT mód (esc: kilépés INSERT módból)
- ZZ kilépés mentéssel
- :qa! kilépés mentés nélkül
- dd sor törlése

leírások:

- http://wiki.math.bme.hu/view/Vim
- http://www.vim.org
- http://vimdoc.sourceforge.net
- vim-ből:
  - :help → lefelé görgetve
  - :help quickref;
  - :help usr_01.txt

hellow.c

```
#include <stdio.h>
int main()
{
   printf("Hello World!\n");
}
```

Fordítás és futtatás

- Fordítás:
  - gcc forrás.c → *a.out*
  - gcc forrás.c -o cél → *cél*
- Futtatás:
  - ***./cél*** vagy
  - ***cél***, ha a PATH környezeti változó elejére hozzáadjuk a . elérési utat (.profile)

Információforrások

- Elsődleges információforrás:
  - man
  - a szükséges include-ok is a man-ból tudhatóak meg
  - a diákon a nem triviális manuál hivatkozások külön jelölve vannak (pl. man printf.3)
- Másodlagos információforrás:
  - Canvas gyakorlati anyagok: gy0.zip..gy6.zip
- Harmadlagos információforrás:
  - gyakorlatok diasorai

man használata

- Parancssorból:
  - man *utasítás*: az *utasítás* manuálját jeleníti meg
    Pl.
    - man scanf → a scanf C nyelvű utasítás leírása
    - man printf → a printf parancsori (pl. bash) utasítás leírása
    - man printf.3 → a printf C nyelvű utasítás leírása
  - man -k *kulcsszó*: a *kulcsszó*t tartalmazó manuálok listája
- VIM-ből:

  Parancsmódban a [shift]+[k] billentyűkombináció a kurzor alatti utasítás triviális manuálját nyitja meg. Ha parancssori a triviális, mint a printf esetében, akkor haszontalan, de pl. a scanf esetén használható.

Canvas gyakorlati anyagok

Előadás canvas

Gyakorlati anyagok

gy0.zip..gy6.zip

Letölteni, kicsomagolni, szerverre feltölteni, elemezni, futtatni, megérteni, módosítgatni!

Gyakorlatok diasorai

- Elérés:

  klettner.web.elte.hu

  Oktatás

  Operációs rendszerek

  Tagozat

- Utasítások kivonatos, leegyszerűsített, megértést segítő, de nem teljeskörű ismertetése (a szakszerű a manuálban található).
- Annak ismertetése, hogy a gy0.zip..gy6.zip tömörített állományokban található egyes mintaprogramok mely utasítások/funkciók működésének megértését segítik.

Házi feladat

- Oldja meg az 1. gyakorló feladatsort (stringek)!
- A 2. gyakorlati diasor (Konzol használat, dinamikus memóriakezelés, paraméterátadás, véletlenszám, idő, fájl- és könyvtártulajdonságok) alapján elevenítse fel és egészítse ki a C programozási nyelvi tudását!
- Oldja meg a 2. gyakorló feladatsort!

(a gyakorló feladatsorokat és a gyakorlati diasorokat a klettner.web.elte.hu weboldalon találja)

Operációs rendszerek

2. gyakorlat

Konzol használat, dinamikus memóriakezelés, paraméterátadás, véletlenszám, idő, fájl- és könyvtártulajdonságok

stringek

A C nyelvben a stringeket karakterek tömbjeként tároljuk.

A string tárolására szolgáló tömb mérete lehet nagyobb, mint a string valós hossza, ezért a stringeket null-terminált karaktersorozatként kezeljük, vagyis a végét egy null-karakter (0-s kódú karakter) jelzi. A lezáró null-karakter nem számít bele a string hosszába, de a string tárolásához szükséges memóriaméretbe igen.

pl. Az "alma" string mérete 4 byte, de a tárolásához minimum 5 byte összefüggő memóriaterület szükséges.

stringek

- null-terminált karaktersorozat:

0	1	2	3	4	5	6	7	8	…
H	a	j	r	á	␣	F	r	a	…

értéke: "Hajrá Fra…

`str[5] = '\0'` vagy `str[5] = 0;`

0	1	2	3	4	5	6	7	8	…
H	a	j	r	á	'\0'	F	r	a	…

értéke: "Hajrá"

string műveletek

- Mivel nincs string típus, így string műveletek sincsenek.
- A string-ekkel történő manipulációk megkönnyítésére a <string.h> és a <strings.h> include-ja szükséges.
  ld.: man string

Dinamikus memóriafoglalás

Ha olyan memóriaterületre van szükségünk, melynek méretét a program írásakor előre nem ismerjük, vagy megváltozhat futásidőben, akkor dinamikus memóriafoglalást alkalmazunk. A dinamikusan foglalt memóriatartomány mindig összefüggő, és kezdőcíme bármely beépített változótípus számára megfelelő. A cast-olásra csak szintaktikai okokból lehet szükség, így az elhagyható. A kezdőcímet mindenképp tárolni kell, mert a méret módosításhoz valamint a lefoglalt terület felszabadításához szükség van rá!

- *pointer változó* = (*típus**)**malloc**(*memóriaméret*)
  Lefoglal *memóriaméret* byte méretű összefüggő memóriaterületet.
  Értéke: A lefoglalt memóriatartomány kezdőcíme, vagy hiba esetén NULL (pointer).

Dinamikus memóriafoglalás

- **free**(mutató)
  Felszabadítja a *mutató* kezdőcímű foglalt memóriaterületet.

- *pointer változó* =
  (*típus**)**realloc**(*mutató*, *memóriaméret*)

  Módosítja a *mutató* kezdőcímű foglalt memória méretét *memóriaméret* méretűre, az eredeti tartalom értelemszerű megtartásával.

  Értéke: A lefoglalt memóriatartomány új kezdőcíme, vagy NULL (hiba).

string deklaráció

- statikus deklaráció: karaktertömbként
  pl.
  - `char str[80]`
    max. 79 byte hosszú string számára, kezdőtartalom nélkül
  - `char str[80]="Hajrá Honvéd!"`
    max. 79 byte hosszú string számára, kezdőtartalommal
  - `char str[]="Hajra Vasas!"`
    13 byte méretű tömb

- dinamikus deklaráció:
  dinamikus memóriafoglalással
  pl.
  - `char *str=malloc(80)`
    `kezdetben` max. 79 byte hosszú string számára, de futás közben változtatható kapacitással, kezdőtartalom nélkül

Paraméterek feldolgozása

A programban az indításkor átadott paramétereket a main függvény paramétereiként tudjuk elérni.

int main(int argc, char **argv)

ahol:

- argc: paraméterek száma
- argv: paraméterek tömbje (string-ek tömbje)

argv elemei:

- **argv[0]**:
  a futtatáshoz beírt parancs
  például: ./a.out, vagy ../gy0/a.out
- **argv[1]**..**argv[argc-1]**:
  a paraméterek a megadásuk sorrendjében
- **argv[argc]**:
  null

Konzolra írás

- **printf**(*formátumstring* [, *értéklista*])
  A konzolra írja a *formátumstring*-et. Ahol a *formátumstring-ben* escape szekvencia vagy formátumleíró áll, ott az annak megfelelő behelyettesítést alkalmazza. Az egyes formátumleírók helyére az értéklista elemeit helyettesíti be rendre a megadásuk sorrendjében. Az értékeket nem a típusuk szerint, hanem a formátumleíró által meghatározott típus szerint dekódolva jeleníti meg.

Konzolra írás

A leggyakrabban használt escape szekvenciák és formátumleírók az alábbi listákban szerepelnek. A formátumleírók pontos és teljes leírása a manuálban található (man printf.3).

escape szekvenciák:

- **\n – új sor**
- **\t - tabulátor**
- **\” – idézőjel**
- **\’ – aposztróf**
- **\0 – null karakter**
- **\r – kurzor a sor elejére**
- **\\ – backslash**

printf formátumleírók:

- **%i – egész**
- **%d – decimális**
- **%f – valós**
- **%c – karakter**
- **%s – string**

Konzolra írás

Fontos tudni, hogy a printf nem közvetlenül ír a konzolra, hanem egy átmeneti tárolóba (puffer) helyezi a kiírandókat, és azok csak akkor jelennek meg ténylegesen a konzolon, amikor valami okból a puffer ürítésére kerül sor. Alaphelyzetben (line buffered) ilyen ok például valamely sorzáró (\n vagy \r) karakter érkezése a pufferbe. (ld. man setbuf)

Erre különösen a párhuzamos folyamatok programozása során, a hibakeresésnél kell figyelni, amikor az egyes események bekövetkezésének sorrendjét kívánjuk követhetővé tenni kiíratásokkal.

Konzolról olvasás

- **scanf**(*formátumstring*, *változó-kezdőcím lista*)
  - A *formátumstring* formátumleírók sorozatát tartalmazza. Minden a *formátumstring*-ben szereplő formátumleíróhoz tartoznia kell a *változó-kezdőcím listá*-ban egy vele azonos sorszámú pointernek. A scanf annyi szöveget olvas a konzolról, ahány formátumleíró szerepel a *formátumstring*-ben.
    Minden egyes beolvasott szöveget átalakít a vele azonos sorszámú formátumleírónak megfelelő típusú értékké, majd elhelyezi *változó-kezdőcím lista* azonos sorszámú eleme által meghatározott memóriacímen. Tehát nem "változóba olvas" a hagyományos értelemben, hiszen a szövegből való értékké alakítás során nem azt veszi figyelembe, hogy milyen típusú a változó amiben a beolvasott értéket tárolnia kell, hanem azt, hogy a formátumleíró milyen típust határoz meg.
  - Szöveg típus beolvasásakor a lezáró null-karaktert is elhelyezi az utolsó beolvasott karakter után.

Konzolról olvasás

- **scanf**(*formátumstring*, *változó-kezdőcím lista*)
  - Ha szöveget kívánunk beolvasni, akkor biztosítani kell a megfelelő méretű karaktertömböt a tárolásához. Mivel a felhasználót nem tudjuk megakadályozni abban, hogy hosszabb szöveget gépeljen be, mint amekkora kapacitást lefoglaltunk a tárolására, gondoskodni kell róla, hogy leálljon a beolvasás a kapacitáshatár elérésekor. Ezt a formátumleíróban a *méret* értékének megadásával tudjuk megtenni. A *méret* értéke a megszakításig beolvasott byte-ok maximális számát határozza meg, de a lezáró null-karakterrel nem számol, így a *méret*-nek 1-gyel kisebbnek kell lennie, mint a rendelkezésre álló tényleges kapacitás. "Túlgépelés" esetén a megszakítás miatt ki nem olvasott szövegtöredék a pufferben marad. Ha szeretnénk elkerülni, hogy ez a következő olvasási műveletbe bezavarjon, gondoskodni kell a pufferből való eltávolításáról.
  - Több szóból álló szöveg beolvasásakor problémát okoz, hogy a whitespace karakterek lezárják a beolvasást, így a szóköz is. Lehetőség van [^ és] jelek közé zárva megadni a beolvasás számára tiltott karaktereket. Ha megadjuk, akkor ezek, és csak ezek a karakterek lesznek a nem beolvasható, vagyis a beolvasást záró karakterek. Praktikusan tehát a new-line karakter megadásával mindent be fog olvasni az enter lenyomásáig. A new-line viszont a pufferben marad, így ennek eltávolításáról itt is külön kell gondoskodni.

Konzolról olvasás

A leggyakrabban használt formátumleírók az alábbi listában szerepelnek.

<u>scanf formátumleírók:</u>

- **%i** – **egész**
- **%d** – **decimális**
- **%f** – **valós**
- **%c** – **karakter**
- **%s** – **string**
- **%*méret*s** – **legfeljebb *méret* byte hosszúságú string (white-space zárja a bevitelt)**
- **%*méret*[^\n]s** – **legfeljebb *méret* byte hosszúságú string (soremelés zárja a bevitelt, de nem kerül kiolvasásra)**

Memóriatartalom mozgatása

- **memcpy**: Egy kezdőcímmel és mérettel (byte) megadott memóriaterület tartalmát átmásolja egy másik, szintén a kezdőcímével megadott memóriaterületre. Egymenetes, így egymást átfedő memóriaterületek esetén nem használható.
- **memmove**: Feladata megegyezik a memcpy-vel, de kétmenetes, így egymást átfedő memóriaterületek esetén is használható.

Véletlenszám

A rand() értéke egy [0,RAND_MAX] intervallumba eső véletlenszerű egész. Tehát a rand() egy folyamaton belül többször lekérdezve minden esetben más-más értékű, vagyis a kapott értéksorozat véletlenszerű számsor, de ugyanarról a kezdőértékről indítva minden futtatáskor ugyanazt a számsorozatot kapjuk. Ha azt szeretnénk, hogy minden futtatás során más-más számsorozatot kapjunk, olyan kezdőértéket kell beállítanunk a rand() számára, ami minden futtatáskor más. Erre elsősorban a rendszeridő vagy a folyamatazonosító (PID) értéke alkalmas.

Véletlenszám

- **rand**(): [0,RAND_MAX] intervallumba eső véletlenszerű egész. (man rand.3)
- **rand**()%B: [0,B-1] intervallumba eső véletlenszerű egész.
- **rand**()%(B-A+1)+A: [A,B] intervallumba eső véletlenszerű egész.
- **srand**(*kezdőérték*): a véletlenszámgenerátor kezdőértékének megadása.
- **getpid**(): folyamatazonosító.

Idő

- **time**(): az aktuális rendszeridő lekérdezése *time_t* típusú értékként (1970-01-01 00:00:00 óta eltelt másodpercek száma)
- **ctime**: kötött formátumú szöveggé konvertálja a *time_t* típusú időértéket (végén soremeléssel).

```
Sat Feb 25 22:39:03 2023
```

- **localtime**: *tm* típusú srtuktúrává konvertálja a *time_t* típusú időértéket.

Idő

- struct **tm** adattagjai:
  - **tm_sec** (int) másodperc (0-61)
  - **tm_min** (int) perc (0-59)
  - **tm_hour** (int) óra (0-23)
  - **tm_mday** (int) nap (1-31)
  - **tm_mon** (int) január óta eltelt hónap (0-11)
  - **tm_year** (int) 1900 óta eltelt év
  - **tm_wday** (int) vasárnap óta eltelt nap (0-6)
  - **tm_yday** (int) Január 1 óta eltelt napok száma (0-365)
  - **tm_isdst** (int) nyári időszámítás flag
    (1: nyári, 0: téli, -1 ismeretlen DST állapot)

gy0.zip és gy1.zip

A következő diákon néhány a gy0.zip és gy1.zip fájlokban található mintaprogram ismertetésére kerül sor.

Az egyes diákon a dia címében feltüntetett nevű, C nyelvű mintafájlban bemutatott funkciók és megoldások felsorolása szerepel.

strings.c (gy0)

- String:
  - nullterminált karaktersorozat
  - statikus és dinamikus deklarációk
  - műveletek (string.h)
    - **strcpy**, **strcat**, **strlen**, **strcmp**, …
- Dinamikus memóriafoglalás
- printf használata (man printf.3)

arg.c (gy0) és write.c (gy1)

- Paraméterátadás:
  - argc: paraméterek száma
  - argv: paraméterek tömbje
- printf használata
- szöveg → szám konverzió (atoi)

rand.c (gy1)

- Véletlenszámok sorozatának előállítása

mtime.c (gy0)

- Fájl adatainak lekérdezése
  - **stat** (man stat.2)
- Idő

passwd.c (gy0)

- Fájl adatainak lekérdezése
  - **stat**
- Felhasználó adatainak lekérdezése
  - **getpwuid**

directory.c (gy0)

- Könyvtár bejárás:
  - **opendir**
  - **readdir**
  - **closedir**
- Fájl adatainak lekérdezése
  - **stat**